

Assignment #8: 🌲 (2/3)

Updated 2223 GMT+8 Oct 27, 2025

2025 fall, Compiled by 窦兆文 元培学院

说明：

1. 解题与记录：

对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge，Codeforces，LeetCode等平台上获得Accepted）。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora <https://typoraio.cn> 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。

2. **提交安排：**提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的本人头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。

3. **延迟提交：**如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

1. 题目

E108.将有序数组转换为二叉搜索树

<https://leetcode.cn/problems/convert-sorted-array-to-binary-search-tree/>

思路：

- 1.选择中间元素作为根节点，这样可以保证左右子树节点数量相近
- 2.递归构建左右子树

代码：

```

class Solution(object):
    def sortedArrayToBST(self, nums):
        """
        :type nums: List[int]
        :rtype: Optional[TreeNode]
        """
        if not nums:
            return None
        mid = len(nums) // 2
        root = TreeNode(nums[mid])
        root.left = self.sortedArrayToBST(nums[:mid])
        root.right = self.sortedArrayToBST(nums[mid+1:])

        return root

```

代码运行截图（至少包含有"Accepted"）



M07161: 森林的带度数层次序列存储

tree, <http://cs101.openjudge.cn/practice/07161/>

思路：

带度数的层次序列：按层次遍历顺序存储，每个节点后跟其度数

后根遍历：对于树来说，先遍历所有子树，最后访问根节点

对于森林：依次对每棵树进行后根遍历

代码：

```

class TreeNode:
    def __init__(self, value):
        self.value = value
        self.children = []

def build_tree_from_sequence(sequence):
    if not sequence:
        return None

    tokens = sequence.split()
    if len(tokens) == 0:
        return None
    root = TreeNode(tokens[0])
    degree = int(tokens[1])
    queue = [(root, degree)]
    idx = 2
    while queue and idx < len(tokens):
        parent, parent_degree = queue.pop(0)
        for _ in range(parent_degree):
            if idx >= len(tokens):
                break

            child_value = tokens[idx]
            child_degree = int(tokens[idx + 1])
            child = TreeNode(child_value)
            parent.children.append(child)
            if child_degree > 0:
                queue.append((child, child_degree))

            idx += 2

    return root

def post_order_traversal(root):
    if root is None:
        return []

    result = []
    for child in root.children:
        result.extend(post_order_traversal(child))
    result.append(root.value)

    return result

```

```
def main():
    n = int(input())
    forest_result = []

    for _ in range(n):
        sequence = input().strip()
        root = build_tree_from_sequence(sequence)
        if root:
            traversal = post_order_traversal(root)
            forest_result.extend(traversal)
    print(' '.join(forest_result))

if __name__ == "__main__":
    main()
```

代码运行截图（至少包含有"Accepted"）

#50683568提交状态

状态: **Accepted**

源代码

M27928: 遍历树

adjacency list, dfs, <http://cs101.openjudge.cn/practice/27928/>

思路:

首先需要构建树结构

找到根节点（没有父节点的节点）

对每个节点，将自己和所有子节点放在一起排序后遍历

代码:

```

class TreeNode:
    def __init__(self, value):
        self.value = value
        self.children = []

def custom_traversal(node):
    if node is None:
        return []

    result = []
    nodes_to_sort = [(node.value, node)]
    for child in node.children:
        nodes_to_sort.append((child.value, child))
    nodes_to_sort.sort(key=lambda x: x[0])
    for value, n in nodes_to_sort:
        if n == node:
            result.append(node.value)
        else:
            result.extend(custom_traversal(n))

    return result

def main():
    n = int(input())
    nodes = {}
    has_parent = set()
    for _ in range(n):
        line = list(map(int, input().split()))
        parent_value = line[0]
        if parent_value not in nodes:
            nodes[parent_value] = TreeNode(parent_value)
        parent = nodes[parent_value]
        if len(line) > 1:
            children_values = line[1:]
            for child_value in children_values:
                if child_value not in nodes:
                    nodes[child_value] = TreeNode(child_value)
                child = nodes[child_value]
                parent.children.append(child)
                has_parent.add(child_value)
    root_value = None
    for value in nodes:
        if value not in has_parent:

```

```

        root_value = value
        break
    if root_value is not None:
        result = custom_traversal(nodes[root_value])
        for val in result:
            print(val)

if __name__ == "__main__":
    main()

```

代码运行截图（至少包含有"Accepted"）

	题目	排名	状态	提问
--	----	----	----	----

#50683596提交状态

状态: **Accepted**

源代码

M129.求根节点到叶节点数字之和

dfs, <https://leetcode.cn/problems/sum-root-to-leaf-numbers/>

思路:

- 1.从根节点开始，逐层累积数字（当前数字 = 父节点数字 × 10 + 当前节点值）
- 2.到达叶节点时，将该路径的数字加入总和
- 3.使用深度优先搜索（DFS）遍历所有路径

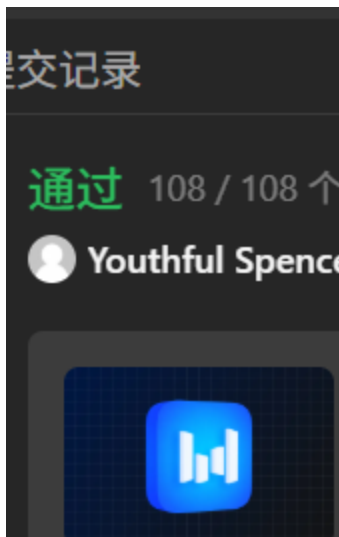
代码

```

class Solution(object):
    def sumNumbers(self, root):
        """
        :type root: Optional[TreeNode]
        :rtype: int
        """
        def dfs(node, current_num):
            if not node:
                return 0
            current_num = current_num * 10 + node.val
            if not node.left and not node.right:
                return current_num
            return dfs(node.left, current_num) + dfs(node.right, current_num)

```

代码运行截图（至少包含有"Accepted"）



M24729: 括号嵌套树

dfs, stack, <http://cs101.openjudge.cn/practice/24729/>

思路:

单个字母表示只有根节点的树

树根(子树1,子树2,...)表示有子树的树

需要递归解析括号内的子树

代码

```

class TreeNode:
    def __init__(self, value):
        self.value = value
        self.children = []

def parse_tree(s):
    if not s:
        return None
    if len(s) == 1:
        return TreeNode(s)
    root = TreeNode(s[0])
    if len(s) > 1 and s[1] == '(':
        children_str = s[2:-1]
        children = split_subtrees(children_str)
        for child_str in children:
            child_node = parse_tree(child_str)
            if child_node:
                root.children.append(child_node)

    return root

def split_subtrees(s):
    subtrees = []
    current = ""
    depth = 0

    for char in s:
        if char == ',' and depth == 0:
            if current:
                subtrees.append(current)
                current = ""
            else:
                current += char
            if char == '(':
                depth += 1
            elif char == ')':
                depth -= 1

        if current:
            subtrees.append(current)

    return subtrees

```

```

def preorder_traversal(node):
    if node is None:
        return []

    result = [node.value]
    for child in node.children:
        result.extend(preorder_traversal(child))

    return result

def postorder_traversal(node):
    if node is None:
        return []

    result = []
    for child in node.children:
        result.extend(postorder_traversal(child))
    result.append(node.value)

    return result

def main():
    tree_str = input().strip()
    root = parse_tree(tree_str)
    preorder = preorder_traversal(root)
    print(''.join(preorder))
    postorder = postorder_traversal(root)
    print(''.join(postorder))

if __name__ == "__main__":
    main()

```

代码运行截图（至少包含有"Accepted"）

#50683623提交状态

状态: **Accepted**

源代码

T02775: 文件结构“图”

tree, <http://cs101.openjudge.cn/practice/02775/>

思路：

代码：

代码运行截图（至少包含有"Accepted"）

2. 学习总结和个人收获

如果发现作业题目相对简单，有否寻找额外的练习题目，如“数算2025fall每日选做”、LeetCode、Codeforces、洛谷等网站上的题目。

通过本周的学习，我认识了树的多种表示方法：掌握了带度数的层次序列、括号嵌套表示、缩进表示等不同的树存储和表示方式，理解了它们之间的转换原理。明白了树的遍历策略：深入理解了前序遍历（根-左-右）、后序遍历（左-右-根）以及自定义遍历规则，认识到遍历本质是递归访问节点的顺序控制。